МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1
по дисциплине «Объектно-ориентированное программирование»
Тема: Создание классов

Студент гр. 3344 _____ Мурдасов М.К.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург
2024

Цель работы

Освоение принципов ООП. Создание классов для корабля, менеджера кораблей и игрового поля в рамках разработки игры “Морской бой”.

Задание

Лабораторная работа №1 - Создание классов

a) Создать класс корабля, который будет размещаться на игровом поле. Корабль может иметь длину от 1 до 4, а также может быть расположен вертикально или горизонтально. Каждый сегмент корабля может иметь три различных состояния: целый, поврежден, уничтожен. Изначально у корабля все сегменты целые. При нанесении 1 урона по сегменту, он становится поврежденным, а при нанесении 2 урона по сегменту, уничтоженным. Также добавить методы для взаимодействия с кораблем.

b) Создать класс менеджера кораблей, хранящий информацию о кораблях. Данный класс в конструкторе принимает количество кораблей и их размеры, которые нужно расставить на поле.

c) Создать класс игрового поля, которое в конструкторе принимает размеры. У поля должен быть метод, принимающий корабль, координаты, на которые нужно поставить, и его ориентацию на поле. Корабли на поле не могут соприкасаться или пересекаться. Для игрового поля добавить методы для указания того, какая клетка атакуется. При попадании в сегмент корабля изменения должны отображаться в менеджере кораблей.

Каждая клетка игрового поля имеет три статуса:

i)неизвестно (изначально вражеское поле полностью неизвестно),

ii)пустая (если на клетке ничего нет)

iii)корабль (если в клетке находится один из сегментов корабля).

Для класса игрового поля также необходимо реализовать конструкторы копирования и перемещения, а также соответствующие им операторы присваивания.

Примечания:

● Не забывайте для полей и методов определять модификаторы доступа

● Для обозначения переменной, которая принимает небольшое ограниченное количество значений, используйте enum

● Не используйте глобальные переменные

- При реализации копирования нужно выполнять глубокое копирование
- При реализации перемещения, не должно быть лишнего копирования
- При выделении памяти делайте проверку на переданные значения
- У поля не должно быть методов возвращающих указатель на поле в явном виде, так как это небезопасно

Выполнение работы

Для начала были реализованы следующие *enum* и структуры:

enum class SegmentState {Intact, Damaged, Destroyed}; - переменная для определения состояния сегмента корабля, где *Intact - 2hp, Damaged - 1hp и Destroyed - 0hp*.

enum class Orientation {Horizontal, Vertical}; - переменная для определения положения корабля на поле.

enum class CellState {Unknown, Empty, Ship}; - переменная для определения статуса клетки на поле, где *Unknown* - неизвестная клетка, *Empty* - пустая клетка, *Ship* - сегмент корабля.

enum class Length {Small = 1, Medium = 2, Large = 3, ExtraLarge = 4}; - переменная для определения длины корабля.

struct Coords {int x, int y}; - структура, хранящая значения координат клетки.

Класс Ship

Описание:

Класс *Ship* является представлением корабля. Он управляет информацией о длине корабля, его ориентации и сегментах, скрывая внутренние детали этой логики. Внешний код не имеет прямого доступа к информации, хранящейся в классе, а может взаимодействовать с кораблем только через специально предусмотренные методы, которые контролируют процесс повреждения сегментов и доступ к информации. Это обеспечивает безопасное взаимодействие между другими компонентами игры.

Поля класса:

- *Length length;* - хранит длину корабля.
- *Orientation orientation;* - хранит ориентацию корабля.
- *std::vector<SegmentState> state*; - хранит состояние сегментов корабля.
- *bool isPlaced*; - true, если корабль размещен на поле

Методы класса:

- **Конструктор:**
    - *Ship(Length length)* - инициализирует объект корабля заданной длины.
- **Метод нанесения урона:**
    - *void hitSegment(int segmentIndex);* - наносит единицу урона выбранному сегменту.
- **Методы доступа к данным:**
    - *void setOrientation(Orientation orientation);* - задает ориентацию корабля.
    - *Length getLength() const;* - возвращает длину.
    - *Orientation getOrientation() const;* - возвращает ориентацию.
    - *SegmentState getSegmentState(int segmentIndex) const;* - возвращает состояние сегмента
    - *void setPlacedStatus(bool placed);* - задает статус размещения
    - *bool getPlacedStatus() const;* - возвращает статус размещения
- **Информационные методы:**
    - *bool isDestroyed() const;* - true, если уничтожен.
- **Проверочные методы (private)**:
    - *bool checkSegmentIndex(int segmentIndex) const;* - проверяет корректность индекса сегмента.

Таким образом, класс *Ship* предоставляет необходимые инструменты управления и работы с состоянием корабля, сохраняя инкапсуляцию.

Класс ShipManager

Описание:

Класс *ShipManager* отвечает за управление всеми кораблями в игре. Он хранит информацию о состоянии и местоположении каждого, тем самым предоставляя контроль над всеми кораблями из одного модуля, централизуя управление.

Поля класса:

std::vector<Ship> ships; - хранит корабли.

Методы класса:

- **Конструктор:**
  - *ShipManager(int numberOfShips, std::vector<Length> lengthsOfShips);* - инициализирует список кораблей, основываясь на их количесве и размерах.
- **Методы управления данными:**
  - *void clearManager();* - очищает менеджер.
- **Методы доступа к данным:**
  - *std::vector<Ship>& getShips();* - возвращает ссылку на массив кораблей, находящихся в игре.
- **Информационные методы:**
  - *bool allShipsDestroyed() const;* - true, если все корабли уничтожены.

Таким образом, класс *ShipManager* объединяет все корабли, находящиеся в игре, сосредотачивая управление над ними в одном модуле и упрощая взаимодействие между классами.

Класс Cell

Описание:

Класс *Cell* представляет собой клетку игрового поля. Он хранит информацию о состоянии и расположении клетки, а также связи с кораблем, если на клетку попадает корабль.

Поля класса:

- *CellState state;* - хранит информацию о состоянии клетки
- *Coords coords;* - хранит информацию о расположении клетки на поле
- *Ship* shipPtr;* - хранит ссылку на корабль
- *unsigned int segmentIndex;* - хранит индекс сегмента корабля

Методы класса:

- **Конструкторы:**
  - *Cell() = default;* - используется только при инициализации игрового поля
  - *Cell(int y, int x);* - создает клетку с заданными координатами
- **Методы доступа к информации:**
  - *void setCellState(CellState newState);* - обновляет статус клетки
  - *CellState getCellState() const;* - возвращает статус клетки
  - *void setShipPtr(Ship* ship);* - задает указатель на корабль
  - *Ship* getShipPtr() const;* - возвращает ссылку на корабль
  - *void setSegmentIndex(unsigned int);* - задает индекс сегмента корабля
  - *unsigned int getSegmentIndex() const;* - возвращает индекс сегмента корабля
  - *Coords getCoords() const;* - возвращает координаты клетки

Класс *Cell* сильно упрощает работу и взаимодействие с полем.

Класс GameField

Описание:

Класс *GameField* представляет собой игровое поле. Он управляет состоянием клеток, размещает корабли и обрабатывает атаки на поле.

Поля класса:

- *int Height;* - хранит высоту поля
- *int Width;* - хранит ширину поля
- *std::vector<std::vector<Cell>> field;* - хранит игровое поле
- *ShipManager& manager;* - хранит ссылку на менеджер кораблей

Методы класса:

- **Конструктор:**
  - *GameField(int width, int height, ShipManager& manager);* - создает игровое поле заданной высоты и ширины
- **Конструкторы копирования и перемещения:**
  - *GameField(const GameField& other);* - выполняет глубокое копирование объекта игрового поля.
  - *GameField(GameField&& other) noexcept;* - выполняет перемещение игрового поля из одного объекта в другой.
- **Операторы копирования и перемещения**
  - *GameField& operator=(const GameField& other);* - обеспечивает глубокое копирование поля.
  - *GameField& operator=(GameField&& other) noexcept;* - обеспечивает перемещение поля.

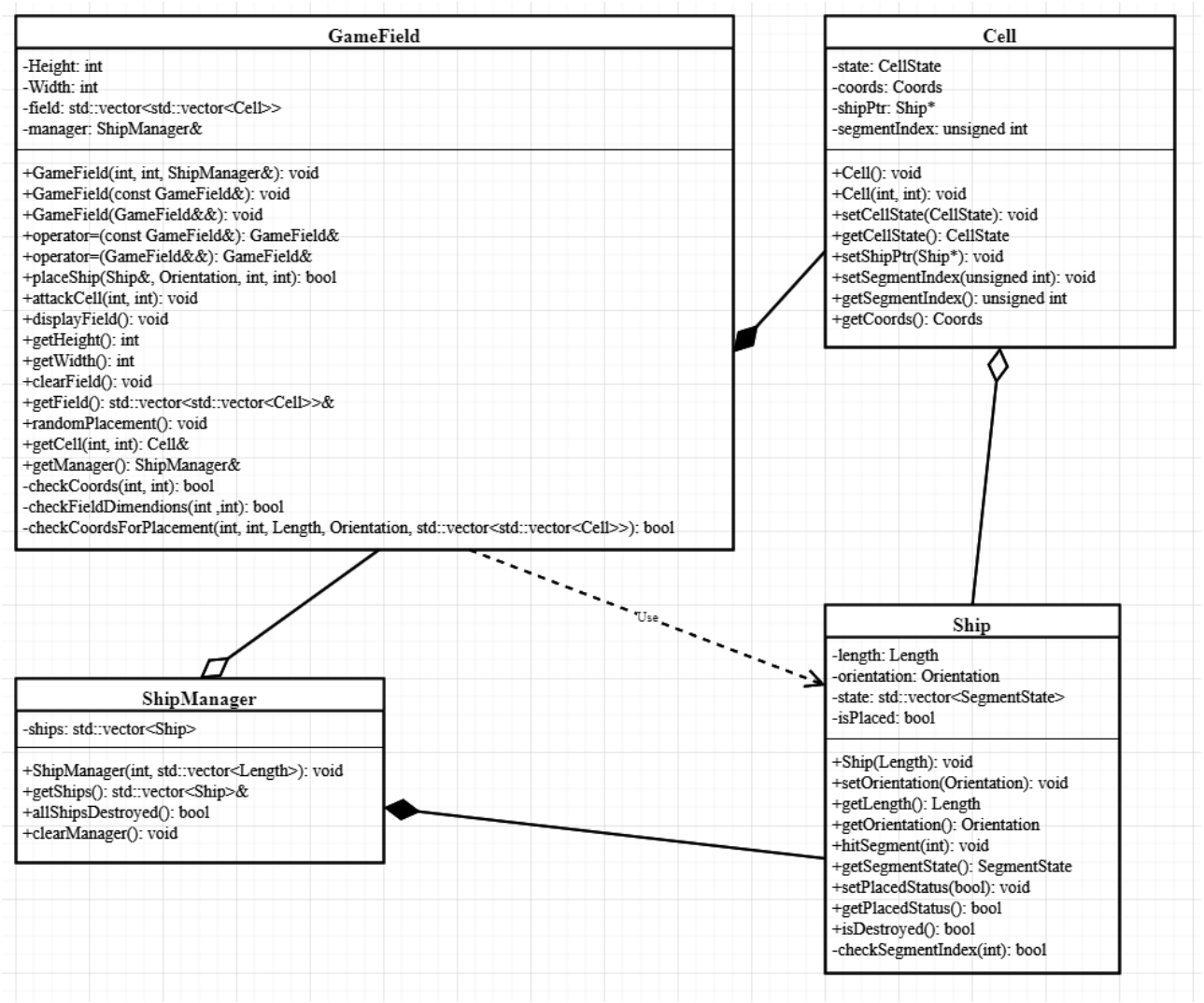
- **Методы управления полем:**
  - *bool placeShip(Ship& ship, Orientation orientation, int startX, int startY);* - размещает корабль на поле
  - *void attackCell(int x, int y);* - наносит удар по выбранной клетке.

- *void displayField() const;* - выводит поле на экран.
- *void clearField();* - очищает поле, обновляя все клетки.
- *void randomPlacement();* - в случайном порядке размещает на поле все корабли из менеджера.

- **Методы доступа к информации:**
  - *int getHeight() const;* - возвращает высоту поля
  - *int getWidth() const;* - возвращает ширину поля
  - *std::vector<std::vector<Cell>>& getField() const;* - возвращает ссылку на поле
  - *Cell& getCell(int x, int y);* - возвращает ссылку на клетку поля
  - *ShipManager& getManager();* - возвращает ссылку на менеджер кораблей
- **Проверочные методы:**
  - *bool checkCoords(int x, int y) const;* - проверяет корректность координат
  - *bool checkFieldDimendions(int width, int height) const;* - проверяет размеры поля на корректность
  - *bool checkCoordsForPlacement(int x, int y, Length len, Orientation orientation, std::vector<std::vector<Cell>> field) const;* - проверяет возможность разместить корабль

Следовательно, *GameField* играет ключевую роль в архитектуре игры, предоставляя возможности для управления клетками, кораблями и их взаимодействиями на игровом поле.

UML – диаграмма реализованных классов



Тестирование

Результаты тестирования представлены в табл. 1.

Таблица 1 – Результаты тестирования

№ п/п	Входные данные	Выходные данные	Комментарии
1.	std::vector<Length> len(2, Length::Large); ShipManager man(2, len); GameField field(5,5,man); field.displayField();	[?] [?]	Корректно
2.	std::vector<Length> len(2, Length::Large); ShipManager man(2, len); GameField field(5,5,man); field.placeShip(man.getShips()[0], Orientation::Horizontal, 0, 0); field.displayField();	[2] [2] [2] [?]	Корректно
3.	std::vector<Length> len(2, Length::Large); ShipManager man(2, len); GameField field(5,5,man); field.placeShip(man.getShips()[0], Orientation::Horizontal, 0, 0); field.attackCell(0,0); field.attackCell(1,0); field.attackCell(1,0); field.displayField();	[1] [X] [2] [?]	Корректно

Вывод:

В процессе выполнения лабораторной работы были изучены основы объектно-ориентированного программирования и создана программа на C++, предназначенная для генерации игрового поля для игры «Морской бой», позволяющая расставлять корабли и осуществлять взаимодействие с ними.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: Cell.cpp

```
#include "Cell.hpp"

Cell::Cell(int y, int x)
    : coords({x, y}), state(CellState::Unknown), shipPtr(nullptr),
segmentIndex(-1) {}

void Cell::setCellState(CellState newState) {
    state = static_cast<CellState>(newState);
}

CellState Cell::getCellState() const {
    return state;
}

Ship* Cell::getShipPtr() const {
    return shipPtr;
}

void Cell::setShipPtr(Ship* ship) {
    shipPtr = ship;
}

void Cell::setSegmentIndex(unsigned int ind) {
    segmentIndex = ind;
}

unsigned int Cell::getSegmentIndex() const {
    return segmentIndex;
}

Coords Cell::getCoords() const {
    return coords;
}
```

Название файла: Cell.hpp

```
#ifndef CELL_H
#define CELL_H

#include "Ship.hpp"

enum class CellState {
    Unknown,
    Empty,
    Ship
};

struct Coords {
    int x;
    int y;
};
```

```
class Cell {
public:
    Cell() = default;
    Cell(int y, int x);
    void setCellState(CellState newState);
    CellState getCellState() const;
    void setShipPtr(Ship* ship);
    Ship* getShipPtr() const;
    void setSegmentIndex(unsigned int);
    unsigned int getSegmentIndex() const;
    Coords getCoords() const;

private:
    CellState state;
    Coords coords;
    Ship* shipPtr;
    unsigned int segmentIndex;
};

#endif CELL_H
```

Название файла: GameField.cpp

```
#include "GameField.hpp"

GameField::GameField(int width, int height, ShipManager& manager)
            :     Width(width),     Height(height),     field(height,
std::vector<Cell>(width)), manager(manager) {
    if (checkFieldDimendions(width, height)) {
        for (int i = 0; i < height; i++) {
            for (int j = 0; j < width; j++) {
                field[i][j] = Cell(i,j);
            }
        }
    }
}

GameField::GameField(const GameField& other)
    : Width(other.Width), Height(other.Height), field(other.field),
manager(other.manager) {}

GameField::GameField(GameField&& other) noexcept
                :      Width(other.Width),      Height(other.Height),
field(std::move(other.field)), manager(other.manager) {
    other.Width = 0;
    other.Height = 0;
}

GameField& GameField::operator=(const GameField& other) {
    if (this != &other) {
        Width = other.Width;
        Height = other.Height;
        field = other.field;
        manager = other.manager;
    }
    return *this;
```

```
    }

    GameField& GameField::operator=(GameField&& other) noexcept {
        if (this != &other) {
            Width = other.Width;
            Height = other.Height;
            manager = other.manager;
            field = std::move(other.field);
            other.Width = 0;
            other.Height = 0;
        }
        return *this;
    }

    bool GameField::checkCoordsForPlacement(int x, int y, Length len,
Orientation orientation, std::vector<std::vector<Cell>> field) const{
        for (int i = 0; i < static_cast<int>(len); i++) {
            int posX = orientation == Orientation::Vertical ? x : x + i;
            int posY = orientation == Orientation::Vertical ? y + i : y;

            if (posX < 0 || posY < 0 || posX >= Width || posY >= Height)
{
                return false;
            }
            if (field[posY][posX].getCellState() != CellState::Unknown)
{
                return false;
            }
        }
        int startX = x;
        int startY = y;
         int endX = orientation == Orientation::Vertical ? x : x +
static_cast<int>(len)-1;
           int endY = orientation == Orientation::Vertical ? y +
static_cast<int>(len)-1 : y;

        std::vector<std::pair<int,int>> badCells;

        if (orientation == Orientation::Vertical) {
            badCells.push_back(std::make_pair(startY - 1, x));
            badCells.push_back(std::make_pair(startY - 1, x - 1));
            badCells.push_back(std::make_pair(startY - 1, x + 1));
            badCells.push_back(std::make_pair(endY + 1, x));
            badCells.push_back(std::make_pair(endY + 1, x - 1));
            badCells.push_back(std::make_pair(endY + 1, x + 1));

            for (int i = 0; i < static_cast<int>(len); i++) {
                badCells.push_back(std::make_pair(startY + i, x + 1));
                badCells.push_back(std::make_pair(startY + i, x - 1));
            }
        } else {

            badCells.push_back(std::make_pair(y, startX - 1));
            badCells.push_back(std::make_pair(y - 1, startX - 1));
            badCells.push_back(std::make_pair(y + 1, startX - 1));
            badCells.push_back(std::make_pair(y, endX + 1));
            badCells.push_back(std::make_pair(y - 1, endX + 1));
            badCells.push_back(std::make_pair(y + 1, endX + 1));
```

```
            for (int i = 0; i < static_cast<int>(len); i++) {
                badCells.push_back(std::make_pair(y + 1, startX + i));
                badCells.push_back(std::make_pair(y - 1, startX + i));
            }
        }

        for(auto& coords : badCells) {
                if (coords.first >= 0 && coords.first < Height &&
coords.second >= 0 && coords.second < Width) {
                if (field[coords.first][coords.second].getCellState() ==
CellState::Ship) {
                    return false;
                }
            }
        }
        return true;
    }

    bool GameField::checkCoords(int x, int y) const {
        if (x < 0 || x >= Width || y < 0 || y >= Height) {
            return false;
        }
        return true;
    }

    bool GameField::checkFieldDimendions(int width, int height) const{
        if (height < 0 || width < 0) {
            return false;
        }
        return true;
    }

    int GameField::getHeight() const {
        return Height;
    }

    int GameField::getWidth() const {
        return Width;
    }

    std::vector<std::vector<Cell>>& GameField::getField(){
        return field;
    }

    void GameField::clearField() {
        for (auto& row : field) {
            row.clear();
        }
        field.clear();
                    std::vector<std::vector<Cell>>      newField(Height,
std::vector<Cell>(Width));
        field = newField;
    }

    bool GameField::placeShip(Ship& ship, Orientation orientation, int
startX, int startY) {
        ship.setOrientation(orientation);
```

```
          if (checkCoordsForPlacement(startX, startY, ship.getLength(),
orientation, field) && ship.getPlacedStatus() == false) {
             for (int i = 0; i < static_cast<int>(ship.getLength()); i++)
{
                int posX = ship.getOrientation() == Orientation::Vertical ?
startX : startX + i;
                int posY = ship.getOrientation() == Orientation::Vertical ?
startY + i : startY;

                 field[posY][posX].setCellState(CellState::Ship);
                 field[posY][posX].setSegmentIndex(i);
                 field[posY][posX].setShipPtr(&ship);
             }
         } else {
             return false;
         }

         ship.setPlacedStatus(true);

         return true;
     }

     void GameField::attackCell(int x, int y) {
         if (checkCoords(x, y)) {
             if (field[y][x].getCellState() == CellState::Ship) {
                  field[y][x].getShipPtr()->hitSegment(field[y][x].getSeg
mentIndex());
             } else {
                 field[y][x].setCellState(CellState::Empty);
             }
         }
     }

     void GameField::randomPlacement() {
         std::random_device rd;
         std::mt19937 gen(rd());

         std::uniform_int_distribution<> randomWidth(0,Width-1);
         std::uniform_int_distribution<> randomHeight(0,Height-1);
         std::uniform_int_distribution<> randomOrientation(0,1);

         for (auto& ship : manager.getShips()) {
             if (ship.getPlacedStatus() == false) {
                 int randomX = randomWidth(gen);
                 int randomY = randomHeight(gen);
                                         Orientation     orientation     =
(Orientation)randomOrientation(gen);
                  while(placeShip(ship, orientation, randomX, randomY) !=
true) {
                     randomX = randomWidth(gen);
                     randomY = randomHeight(gen);
                     orientation = (Orientation)randomOrientation(gen);
                 }
             }
         }
     }

     void GameField::displayField() const {
```

```
        for (int y = 0; y < Height; ++y) {
            for (int x = 0; x < Width; ++x) {
                switch (field[y][x].getCellState()) {
                    case CellState::Unknown:
                       std::cout << "\033[1;34m" << "[?] " << "\033[0m";
                        break;
                    case CellState::Empty:
                        std::cout << "\033[1;34m" << "[" << "\033[0m" <<
"\u25A0" << "\033[1;34m" << "] " << "\033[0m";
                        break;
                    case CellState::Ship:
                                                        int    damaged    =
static_cast<int>(field[y][x].getShipPtr()->getSegmentState(field[y][x].ge
tSegmentIndex()));
                        switch (damaged) {
                            case 0:
                                 std::cout << "\033[1;31m" << "[2] " <<
"\033[0m";
                                break;
                            case 1:
                                 std::cout << "\033[1;31m" << "[1] " <<
"\033[0m";
                                break;
                            case 2:
                                 std::cout << "\033[1;30m" << "[X] " <<
"\033[0m";
                                break;
                        }
                        break;
                }
            }
            std::cout << std::endl;
        }

        std::cout << "\n-----------------------------------------------
---------------------------------------------------------------------------
-----------------------------------\n" <<std::endl;
    }

    Cell& GameField::getCell(int x, int y) {
        return field[y][x];
    }

    ShipManager& GameField::getManager() {
        return manager;
    }
```

Название файла: GameField.hpp

```
#ifndef GAMEFIELD_H
#define GAMEFIELD_H

#include <iostream>
#include <random>
#include <cstdlib>
#include <ctime>
```

```
#include "Cell.hpp"
#include "ShipManager.hpp"

class GameField {
public:
    GameField(int width, int height, ShipManager& manager);
    GameField(const GameField& other);
    GameField(GameField&& other) noexcept;
    GameField& operator=(const GameField& other);
    GameField& operator=(GameField&& other) noexcept;
    bool placeShip(Ship& ship, Orientation orientation, int startX,
int startY);
    void attackCell(int x, int y);
    void displayField() const;
    int getHeight() const;
    int getWidth() const;
    void clearField();
    std::vector<std::vector<Cell>>& getField();
    void randomPlacement();
    Cell& getCell(int x, int y);
    ShipManager& getManager();

private:
    int Height;
    int Width;
    std::vector<std::vector<Cell>> field;
    ShipManager& manager;

    bool checkCoords(int x, int y) const;
    bool checkFieldDimendions(int width, int height) const;
      bool  checkCoordsForPlacement(int  x,  int  y,  Length  len,
Orientation orientation, std::vector<std::vector<Cell>> field) const;
};

#endif GAMEFIELD_H
```

Название файла: Ship.cpp

```
#include "Ship.hpp"

Ship::Ship(Length length)
                                    :                length(length),
state(std::vector<SegmentState>(static_cast<int>(length),
SegmentState::Intact)) {
    setPlacedStatus(false);
}

bool Ship::checkSegmentIndex(int segmentIndex) const {
            if    (segmentIndex    <    0    ||    segmentIndex    >
static_cast<int>(this->length)) {
          return false;
    }
    return true;
}

void Ship::setPlacedStatus(bool placed) {
```

```
    isPlaced = placed;
}

bool Ship::getPlacedStatus() const {
    return isPlaced;
}

void Ship::setOrientation(Orientation orientation) {
    this->orientation = orientation;
}

Length Ship::getLength() const {
    return length;
}

Orientation Ship::getOrientation() const {
    return orientation;
}

SegmentState Ship::getSegmentState(int segmentIndex) const {
    if (checkSegmentIndex(segmentIndex)) return state[segmentIndex];
}

void Ship::hitSegment(int segmentIndex) {
    if (checkSegmentIndex(segmentIndex)) {
        if (state[segmentIndex] == SegmentState::Intact) {
            state[segmentIndex] = SegmentState::Damaged;
        } else if (state[segmentIndex] == SegmentState::Damaged) {
            state[segmentIndex] = SegmentState::Destroyed;
        }
    }
}

bool Ship::isDestroyed() const {
    for (const auto& stateInfo : state) {
        if (stateInfo != SegmentState::Destroyed) {
            return false;
        }
    }
    return true;
}
```

Название файла: Ship.hpp

```
#ifndef SHIP_H
#define SHIP_H

#include <vector>

enum class SegmentState {
    Intact,
    Damaged,
    Destroyed
};

enum class Orientation {
    Horizontal,
```

```
    Vertical
};

enum class Length {
    Small = 1,
    Medium = 2,
    Large = 3,
    ExtraLarge = 4
};

class Ship {
public:
    Ship(Length length);
    void setOrientation(Orientation orientation);
    Length getLength() const;
    Orientation getOrientation() const;
    void hitSegment(int segmentIndex);
    SegmentState getSegmentState(int segmentIndex) const;
    void setPlacedStatus(bool placed);
    bool getPlacedStatus() const;
    bool isDestroyed() const;

private:
    Length length;
    Orientation orientation;
    std::vector<SegmentState> state;
    bool isPlaced;

    bool checkSegmentIndex(int segmentIndex) const;
};

#endif SHIP_H
```

Название файла: ShipManager.cpp

```
#include "ShipManager.hpp"

ShipManager::ShipManager(int   numberOfShips,   std::vector<Length>
lengthsOfShips) {
    for (const auto& len : lengthsOfShips) {
        ships.emplace_back(Ship(len));
    }
}

void ShipManager::clearManager() {
    ships.clear();
}

std::vector<Ship>& ShipManager::getShips() {
    return ships;
}

bool ShipManager::allShipsDestroyed() const {
    for (const auto& ship : ships) {
        if (!ship.isDestroyed()) {
            return false;
        }
```

```
        }
        return true;
    }
```

Название файла: ShipManager.hpp

```
    #ifndef SHIPMANAGER_H
    #define SHIPMANAGER_H
    #include "Ship.hpp"

    class ShipManager {
    public:
                ShipManager(int    numberOfShips,    std::vector<Length>
lengthsOfShips);
        std::vector<Ship>& getShips();
        bool allShipsDestroyed() const;
        void clearManager();

    private:
        std::vector<Ship> ships;
    };

    #endif SHIPMANAGER_H
```